

# tf.Graph Stay organized with collections Save and categorize content based on your preferences.

[https://www.tensorflow.org/api\\_docs/python/tf/Graph](https://www.tensorflow.org/api_docs/python/tf/Graph)

---

*A TensorFlow computation, represented as a dataflow graph.*

## Function Signatures

---

```
tf.Graph() -> None
```

## Code Examples

---

```
tf.Graph() -> None
```

```
tf.Graph() -> None
```

```
g = tf.Graph()
with g.as_default():
    # Define operations and tensors in `g`.
    c = tf.constant(30.0)
    assert c.graph is g
```

```
g = tf.Graph()
with g.as_default():
    # Define operations and tensors in `g`.
    c = tf.constant(30.0)
    assert c.graph is g
```

```
Dismantle()
```

```
Dismantle()
```

```
add_to_collection(
    name, value
) -> None
```

```
add_to_collection(
    name, value
) -> None
```

# Documentation

---

A TensorFlow computation, represented as a dataflow graph.

## View aliases

Compat aliases for migration

See

Migration guide for  
more details.

`tf.compat.v1.Graph`

## Used in the notebooks

- Migrating model checkpoints
  - Validating correctness & numerical equivalence
  - Migrate the SavedModel workflow
  - Debug a TensorFlow 2 migrated training pipeline
  - Migrating your TFLite code to TF2
  - Generating Images with Little Data Using S3GAN
  - Exploring the TF-Hub CORD-19 Swivel Embeddings
  - Wiki40B Language Models
  - Migrating `tf.summary` usage to TF 2.x

Graphs are used by `tf.functions` to represent the function's computations. Each graph contains a set of `tf.Operation` objects, which represent units of computation; and `tf.Tensor` objects, which represent the units of data that

flow between operations.

## Using graphs directly (deprecated)

A `tf.Graph` can be constructed and used directly without a `tf.function`, as was required in TensorFlow 1, but this is deprecated and it is recommended to use a `tf.function` instead. If a graph is directly used, other deprecated TensorFlow 1 classes are also required to execute the graph, such as a `tf.compat.v1.Session`.

A default graph can be registered with the `tf.Graph.as_default` context manager. Then, operations will be added to the graph instead of being executed eagerly. For example:

`tf.compat.v1.get_default_graph()` can be used to obtain the default graph.

Important note: This class is not thread-safe for graph construction. All operations should be created from a single thread, or external synchronization must be provided. Unless otherwise specified, all methods are not thread-safe.

A `Graph` instance supports an arbitrary number of "collections" that are identified by name. For convenience when building a large graph, collections can store groups of related objects: for example, the `tf.Variable` uses a collection (named `tf.GraphKeys.GLOBAL_VARIABLES`) for all variables that are created during the construction of a graph. The caller may define additional collections by specifying a new name.

## Attributes

For details on the meaning of each version, see `GraphDef`.

## Methods

`## Dismantle`

`(self: handle) -> None`

### **`add_to_collection`**

[View source](#)

Stores value in the collection with the given name.

Note that collections are not sets, so it is possible to add a value to a collection several times.

### **`add_to_collections`**

[View source](#)

Stores value in the collections given by names.

Note that collections are not sets, so it is possible to add a value to a collection several times. This function makes sure that duplicates in names are ignored, but it will not check for pre-existing membership of value in any of the collections in names.

names can be any iterable, but if names is a string, it is treated as a single collection name.

## **as\_default**

[View source](#)

Returns a context manager that makes this Graph the default graph.

This method should be used if you want to create multiple graphs in the same process. For convenience, a global default graph is provided, and all ops will be added to this graph if you do not create a new graph explicitly.

Use this method with the `with` keyword to specify that ops created within the scope of a block should be added to this graph. In this case, once the scope of the `with` is exited, the previous default graph is set again as default. There is a stack, so it's ok to have multiple nested levels of `as_default` calls.

The default graph is a property of the current thread. If you create a new thread, and wish to use the default graph in that thread, you must explicitly add a `with g.as_default():` in that thread's function.

The following code examples are equivalent:

If eager execution is enabled ops created under this context manager will be added to the graph instead of executed eagerly.

## **as\_graph\_def**

[View source](#)

Returns a serialized GraphDef representation of this graph.

The serialized GraphDef can be imported into another Graph (using `tf.import_graph_def`) or used with the C++ Session API.

This method is thread-safe.

## **as\_graph\_element**

[View source](#)

Returns the object referred to by `obj`, as an Operation or Tensor.

This function validates that `obj` represents an element of this graph, and gives an informative error message if it is not.

This function is the canonical way to get/validate an object of one of the allowed types from an external argument reference in the Session API.

This method may be called concurrently from multiple threads.

## **clear\_collection**

[View source](#)

Clears all values in a collection.

## **colocate\_with**

[View source](#)

Returns a context manager that specifies an op to colocate with.

### **For example:**

b and c will always be colocated with a, no matter where a is eventually placed.

If op is None then ignore\_existing must be True and the new scope resets all colocation and device constraints.

### **container**

[View source](#)

Returns a context manager that specifies the resource container to use.

Stateful operations, such as variables and queues, can maintain their states on devices so that they can be shared by multiple processes. A resource container is a string name under which these stateful operations are tracked. These resources can be released or cleared with `tf.Session.reset()`.

### **For example:**

```
## control_dependencies
```

[View source](#)

Returns a context manager that specifies control dependencies.



Use with the `with` keyword to specify that all operations constructed within the context should have control dependencies on `control_inputs`. For example:

Multiple calls to `control_dependencies()` can be nested, and in that case a new `Operation` will have control dependencies on the union of `control_inputs` from all active contexts.

You can pass `None` to clear the control dependencies:

Also note that though execution of ops created under this scope will trigger execution of the dependencies, the ops created under this scope might still be pruned from a normal tensorflow graph. For example, in the foll